Penetration Testing Engagement: Firmware Security Assessment

Samuel LaTourette, Erek Gilbertson, Jessica Ottor

10-12-2025

ThanosTechLLC

Table of Contents

1. Introduction
    a. Project Overview
    b. Scope and Objectives
2. Methodology
    a. Tools and Techniques
    b. Firmware Analysis Process
    c. Vulnerability Discovery
3. Findings
    a. Critical Vulnerabilities
    b. High-Risk Issues
    c. Impact Assessment
4. Automation Script
    a. Script Overview
    b. Implementation Details
    c. Results and Output
5. Recommendations
    a. Immediate Actions
    b. Long-term Security Measures
6. Conclusion
7. Appendices
    a. Screenshots and Evidence
    b. Technical Outputs

Introduction

ThanosTech LLC is a spaceship components manufacturer that has requested a vulnerability assessment of its firmware systems. The company's goal is to ensure the security and reliability of the embedded software that powers its next-generation space technologies.

The assessment focuses on two Linux-based firmware images and several bare-metal ELF binaries. The Linux firmware includes a D-Link DCS-8000LH image used in ThanosTech's state-of-the-art space IP camera, and a TP-Link TL-WR841N image used in a specialized space router that supports on-board communications. In addition, several bare-metal ELF files are provided, representing the microcontroller firmware used in ThanosTech's advanced control systems.

This report documents the process and findings of the firmware security assessment, highlighting potential vulnerabilities, explaining the analysis methodology, and providing recommendations for improving the overall security posture of ThanosTech's firmware.

Scope

The scope of this project is limited to a static firmware vulnerability assessment. No live exploitation or network-based testing was performed. All firmware samples were provided directly by ThanosTech LLC specifically for vulnerability analysis. The objective is to find and identify security weaknesses, such as hardcoded credentials, outdated components, etc., through analysis and tool-assisted examination.

Methodology

Several tools were used to explore and evaluate the provided firmware files. One of the most important tools was Binwalk, which was used for reconnaissance and extraction. Binwalk was first run on the router firmware image, revealing that it utilized a SquashFS filesystem and a U-Boot bootloader, along with other details. By using the -e flag, the firmware image was extracted to a different directory (Figure 1), providing access to the internal directory structure of the device.

```
sam@bonehead:~/midterm1$ binwalk -e  wr-841n.bin

DECIMAL       HEXADECIMAL     DESCRIPTION
--------------------------------------------------------------------------------
66048         0x10200         LZMA compressed data, properties: 0x5D, dictionary size:
8388608 bytes, uncompressed size: 2986732 bytes

WARNING: Extractor.execute failed to run external extractor 'sasquatch -p 1 -le -d 'squ
ashfs-root-0' '%e'': [Errno 2] No such file or directory: 'sasquatch', 'sasquatch -p 1
-le -d 'squashfs-root-0' '%e'' might not be installed correctly
```

```
WARNING: One or more files failed to extract: either no utility was found or it's unimp
lemented

sam@bonehead:~/midterm1$ ls
bare-metal-demo.elf      dcs-8000lh.bin  _wr-841n.bin.extracted
bare-metal-takehome.elf  wr-841n.bin
sam@bonehead:~/midterm1$ cd _wr-841n.bin.extracted/S
```

Figure 1: Binwalk reveals a SquashFS filesystem, U-Boot bootloader.

Methodology (continued)

Another important set of tools used in the analysis were file, hexdump, and grep. The file command allowed us to view information about the bare-metal ELF files, including that they are compiled for ARM architecture (Figure 2). The hexdump command was used to display raw binary data in a human-readable format, while grep allowed for searching specific strings or character patterns within that data. By combining these two tools on the bare-metal-takehome-elf file (Figure 2), we were able to identify sensitive information, including two AWS keys, an RSA private key, and GitHub access tokens (GHO and GHP).

```
└─$file bare-metal-takehome.elf
bare-metal-takehome.elf: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV),
```

Figure 2: File command showing ARM architecture of bare-metal files.

```
┌──(user㉿parrot)-[~]
└─$hexdump bare-metal-takehome.elf -C | grep aws
00001170  75 6c 74 5d 5c 6e 61 77  73 5f 61 63 63 65 73 73  |ult]\naws_access|
000011a0  61 77 73 5f 73 65 63 72  65 74 5f 61 63 63 65 73  |aws_secret_acces|
┌──(user㉿parrot)-[~]
└─$
```

Figure 3: Hexdump and grep used to find AWS keys.

Methodology (continued)

While analyzing the router's directory system, we used grep to locate a file named passwd.bak (Figure 4). Further investigation revealed that during each system boot, this password file is copied into a live system path. This behavior suggests the presence of hardcoded credentials and indicates poor password-handling practices within the firmware.

Figure 4: Locating passwd.bak file using grep.

```
ottoramaka@cs-768527768013-default:~/firmware_project/_wr-841n.bin-0.extracted/squashfs-root$ grep -R -n "cp
 " etc/init.d | grep -v "#"
etc/init.d/rcS:15:cp -p /etc/SingleSKU_FCC.dat /var/Wireless/RT2860AP/SingleSKU.dat
etc/init.d/rcS:20:cp -p /etc/passwd.bak /var/passwd
ottoramaka@cs-768527768013-default:~/firmware_project/_wr-841n.bin-0.extracted/squashfs-root$ grep -R -n -E
"cp .*passwd|cp .*conf|cp .*dat|cp .*key|cp .*cfg" .
grep: ./usr/bin/dropbearmulti: binary file matches
grep: ./usr/bin/scp: binary file matches
grep: ./usr/bin/dropbear: binary file matches
grep: ./usr/bin/dropbearkey: binary file matches
./etc/init.d/rcS:15:cp -p /etc/SingleSKU_FCC.dat /var/Wireless/RT2860AP/SingleSKU.dat
./etc/init.d/rcS:20:cp -p /etc/passwd.bak /var/passwd
grep: ./lib/libcmm.so: binary file matches
ottoramaka@cs-768527768013-default:~/firmware_project/_wr-841n.bin-0.extracted/squashfs-root$
```

Upon further investigation of passwd.bak, we used cat to reveal a hardcoded administrative account with root privileges and an associated password hash (Figure 5). The hash appeared to be weak, so we used 'openssl passwd' to test common default passwords and successfully recovered the credentials: username admin and password 1234 (Figure 5). Because the router does not require a credential change on first login, this represents a **high-risk vulnerability** and operational weakness.

```
sam@bonehead:~/midterm1/_wr-841n.bin.extracted/squashfs-root$ openssl passwd -1 -salt '' ''
$1$$qRPK7m23GJusamGpoGLby/
sam@bonehead:~/midterm1/_wr-841n.bin.extracted/squashfs-root$ openssl passwd -1 -salt '' 'admin'
$1$$CoERg7ynjYLsj2j4glJ34.
sam@bonehead:~/midterm1/_wr-841n.bin.extracted/squashfs-root$
sam@bonehead:~/midterm1/_wr-841n.bin.extracted/squashfs-root$ openssl passwd -1 -salt '' 'password'
$1$$I2o9Z7NcvQAKp7wyCTlia0
sam@bonehead:~/midterm1/_wr-841n.bin.extracted/squashfs-root$ openssl passwd -1 -salt '' 'Admin'
$1$$OqnltKIlb1a5NyG.vLqh0/
sam@bonehead:~/midterm1/_wr-841n.bin.extracted/squashfs-root$ openssl passwd -1 -salt '' 'root'
$1$$oCLuEVgI1iAqOA8pwkzAg1
sam@bonehead:~/midterm1/_wr-841n.bin.extracted/squashfs-root$
sam@bonehead:~/midterm1/_wr-841n.bin.extracted/squashfs-root$ openssl passwd -1 -salt '' '1234'
$1$$iC.dUsGpxNNJGeOm1dFio/
sam@bonehead:~/midterm1/_wr-841n.bin.extracted/squashfs-root$
```

Figure 5: Password hash discovery and cracking process using openssl passwd to recover "1234" and hardcoded admin credentials discovered in passwd.bak file.

Methodology (continued)

```
ottoramakafoo-768527768013-default:~/firmware_project/_wr-841n.bin-0.extracted/squashfs-root$ grep -R -n -E "telnetd|dropbear|ssh" . | head -n
30
grep: ./usr/bin/dropbearmulti: binary file matches
grep: ./usr/bin/taskset: binary file matches
grep: ./usr/bin/scp: binary file matches
grep: ./usr/bin/ipcs: binary file matches
grep: ./usr/bin/killall: binary file matches
grep: ./usr/bin/ipcrm: binary file matches
grep: ./usr/bin/free: binary file matches
grep: ./usr/bin/dropbear: binary file matches
grep: ./usr/bin/cos: binary file matches
grep: ./usr/bin/tftp: binary file matches
grep: ./usr/bin/top: binary file matches
grep: ./usr/bin/dropbearkey: binary file matches
grep: ./usr/bin/arping: binary file matches
grep: ./usr/sbin/brctl: binary file matches
grep: ./bin/date: binary file matches
./etc/init.d/rcS:17:/bin/mkdir -m 0777 -p /var/tmp/dropbear
./etc/group:29:sshd:x:74:
./etc/services:45:ssh            22/tcp                          # SSH Remote Login Protocol
./etc/services:46:ssh            22/udp                          # SSH Remote Login Protocol
./etc/services:535:x11-ssh-offset        6010/tcp                        # SSH X11 forwarding offset
./etc/passwd.bak:2:dropbear:x:500:500:dropbear:/var/dropbear:/bin/sh
grep: ./bin/df: binary file matches
grep: ./bin/rm: binary file matches
grep: ./bin/mount: binary file matches
grep: ./bin/umount: binary file matches
grep: ./bin/echo: binary file matches
grep: ./bin/cat: binary file matches
grep: ./bin/ping: binary file matches
grep: ./bin/cp: binary file matches
grep: ./bin/sleep: binary file matches
grep: ./bin/ping6: binary file matches
grep: ./bin/pidof: binary file matches
grep: ./bin/kill: binary file matches
grep: ./bin/busybox: binary file matches
grep: ./bin/ps: binary file matches
grep: ./bin/chmod: binary file matches
grep: ./bin/sh: binary file matches
grep: ./bin/login: binary file matches
grep: ./bin/ash: binary file matches
grep: ./bin/ls: binary file matches
```

Figure 6: World-writable Dropbear directory permissions in rcS script.

While inspecting the startup script `rcS`, we found it creates the Dropbear runtime directory with world-writable permissions, which would allow unprivileged actors to modify or replace the SSH host keys (Figure 6). This is a **serious vulnerability** because it can enable local privilege escalation and **affect the integrity** of SSH sessions by allowing incorrect host keys to be used.

```
┌─[user@parrot]─[~/_wr-841n.bin.extracted/squashfs-root/bin]
└──╼ $binwalk busybox

DECIMAL       HEXADECIMAL     DESCRIPTION
--------------------------------------------------------------------------------
0             0x0             ELF, 32-bit LSB MIPS64 executable, MIPS, version 1 (SYSV
235992        0x399D8         Copyright string: "Copyright (C) 1998-2011 Erik Andersen
251572        0x3D6B4         Unix path: /etc/init.d/rcS

┌─[user@parrot]─[~/_wr-841n.bin.extracted/squashfs-root/bin]
└──╼ $busybox
BusyBox v1.35.0 (Debian 1:1.35.0-4+b7) multi-call binary.
BusyBox is copyrighted by many authors between 1998-2015.
Licensed under GPLv2. See source distribution for detailed
copyright notices.
```

Figure 7: Outdated BusyBox binary version 1.35.0 identified.

Within the `/bin` directory of the router's firmware image, we identified an outdated BusyBox binary (Figure 7). BusyBox is a utility that combines many common Linux commands into a single executable, typically used in embedded systems. The version in the firmware is 1.35.0, which is outdated and contains several publicly known vulnerabilities.

Findings

Multiple significant vulnerabilities were identified.

Critical: Hardcoded Admin Credentials

As shown in Figures 4 and 5, there are hardcoded admin credentials with a weak password hash that will give immediate, full system access to anyone who discovers them. This vulnerability will completely compromise the device. Attackers will be able to log in remotely, extract data, or use the router as a pivot point. This issue aligns with CWE-259 (Use of Hard-coded Password). This vulnerability can be mitigated by removing default passwords, enforcing credential changes on first boot, and implementing industry-standard password complexity rules.

High: World-Writable Dropbear Directory

As shown in Figure 6, there is a world-writable directory that will allow the modification of SSH host keys or the injection of malicious ones. Attackers can perform man-in-the-middle attacks or gain persistent privileged access by replacing host keys. This issue aligns with CWE-732 (Incorrect Permission Assignment for Critical Resource). This vulnerability can be mitigated by restricting permissions, generating host keys securely, and verifying integrity on boot.

High: Outdated BusyBox Binary

As shown in Figure 7, an outdated version of BusyBox(v1.35.0 (2022-08-16 04:14:58) is running on the router firmware image. Version 1.35.0 has multiple known CVEs, especially CVE-2022-48174, which allows for remote code execution, as well as others with privilege escalation. This vulnerability can be mitigated by updating to the latest BusyBox version and rebuilding the firmware.

High: Embedded Private Keys (AWS, RSA, GitHub)

As shown in Figures 2 and 3, there are exposed private or API keys in the bare-metal-takehome.elf file. If these keys are valid, it will give access to cloud services or source code. Mitigation includes immediately revoking exposed keys and using environment variables and secure key storage.

Automation Script

The purpose of our automation script is to make the firmware analysis process faster and more consistent across multiple firmware samples. Instead of manually running tools and searching through extracted files, the script automatically identifies, extracts, and scans files for potential vulnerabilities. The tools used **include** `file`, Binwalk, `find`, `awk`, and Nosey Parker to detect notable issues such as embedded credentials, outdated binaries, password files, and private keys.

The script can be executed from the Linux command line by running “sudo bash fw_triage.sh” and will traverse all files in the directory it is run in, except for the script itself or its subdirectories. The script first checks if Nosey Parker is installed, and if it is not, it prompts the user to install the binary from the GitHub page linked in the README.

The next step is to determine if the file being analyzed is a bare-metal or a Linux-based binary. This is done using the `file` command and checking if it is `data` (meaning a Linux-based binary) or `executable` (meaning it is a bare-metal binary). The bare-metal binaries are then processed by Nosey Parker, and the output of Nosey Parker is piped through `awk`, which extracts data from “finding” to “lines”—the main body of the results—thereby cleaning up the data.

Automation Script (continued)

The next step is to determine if the file being analyzed is a bare-metal-based or a Linux-based binary. This is done using the `file` command and grepping for "data" (indicating a Linux-based binary) or "executable" (indicating a bare-metal-based binary). The bare-metal-based binaries are run through Nosey Parker, and the output of Nosey Parker is then processed by `awk`, which pulls data from "finding" to "lines" (the main body of the results), cleaning up the data.

```
=== Analyzing: bare-metal-takehome.elf ===
Type: bare-metal-takehome.elf: ELF 32-bit LSB executable, ARM, EABI5 version 1 (
SYSV), statically linked, stripped
-> Binary detected. Running noseyparker scan...
Scanning content [00:00:00]
Scanned 6.68 KiB from 1 blobs in 0 seconds (140.92 KiB/s); 5/5 new matches

 Rule                          Findings   Matches
 ---------------------------------------------------
 AWS API Credentials                1         2
 GitHub OAuth Access Token          1         1
 GitHub Personal Access Token       1         1
 PEM-Encoded Private Key            1         1

Run the `report` command next to show finding details.
-> Fetching noseyparker results and extracting metadata...
```

Figure 8: Bare-metal-based binary analysis with Nosey Parker showing multiple results.

Nosey Parker also runs on the Linux-based binary to find any embedded data, but the primary data is collected by using Binwalk, which extracts the filesystem from the binary. Then, several searches using the `find` command locate important potential vulnerabilities in the extracted filesystems. These include possible password files, `/etc/shadow` files, BusyBox versions, init scripts or folders, and private keys. This gives investigators a good grasp of vulnerabilities in the systems, leaves the extracted information for future deeper analysis, and saves valuable time on extraction and simple analysis of the files.

```
=== Analyzing: wr-841n.bin ===
Type: wr-841n.bin: data
-> Linux-related file. Running binwalk...
BusyBox version: BusyBox v1.19.2 (2022-08-16 12:08:08 CST)
Potential password files detected:
_wr-841n.bin.extracted/squashfs-root/etc/passwd.bak
Potential embedded /etc/shadow file detected:
_wr-841n.bin.extracted/squashfs-root/etc/group:root:x:0:root
Potential embedded init script references detected:
_wr-841n.bin.extracted/squashfs-root/etc/inittab:::sysinit:/etc/init.d/rcS
```

Figure 9: Linux-based binary analysis with Binwalk and `find` command results.

Recommendations

Based on our findings, we would recommend the following steps to remediate the vulnerabilities.

Critical Hardcoded Credentials

Remove default passwords, enforce credential changes on first boot, and implement industry-standard password complexity rules.

World-Writable Dropbear Directory

Restrict permissions, generate host keys securely, and verify integrity on boot.

Outdated BusyBox Binary

Update to the latest BusyBox version and rebuild the firmware.

Embedded Private Keys

Immediately revoke exposed keys and use environment variables and secure key storage.

Conclusion

Throughout this analysis, several significant vulnerabilities were discovered within ThanosTech's firmware, including hardcoded administrative credentials, exposed private keys, world-writable directories, and outdated system components. Each of these issues poses a serious risk to the confidentiality, integrity, and availability of the affected systems. The presence of default credentials and weak password handling could allow immediate unauthorized access, while misconfigured permissions and outdated binaries open the door for privilege escalation or remote code execution.

These findings highlight the **importance of firmware hardening** as a core part of secure development. Firmware often serves as the foundation of embedded and IoT devices, and once compromised, it can be extremely difficult to remediate. Regularly updating components, removing hardcoded credentials, validating permissions, and securely managing cryptographic keys are critical steps to maintaining security.

By performing this assessment, we were able to better understand how real-world vulnerabilities can exist even in legitimate firmware releases. Implementing proper security practices early in the firmware development process will greatly reduce the risk of exploitation and help ensure ThanosTech's products remain secure and reliable long-term.

Appendices: Screenshots and Evidence (Part 1)

Full list of screenshots and outputs leading to hardcoded credentials, as well as other noteworthy files.

```
ottoramaka@cloudshell:~/firmware_project$ tree -a > wr841n_tree.txt
grep -R -I -n "password" _wr-841n.bin.extracted/ | head
_wr-841n.bin.extracted/squashfs-root/etc/services:259:shell              514/tcp         cmd             # no passwords used
_wr-841n.bin.extracted/squashfs-root/web/help/RestoreDefaultCfgHelpRpm.htm:25:<script language="javascript" type="text/javascript">window.parent.$.transHelp(document.URL);document.getElementById("defaultUserName").i
nnerHTML+=" - <B>"+(typeof default_user_name!="undefined"?default_user_name:"admin")+"</B>";if(INCLUDE_US_CPI_SPEC){document.getElementById("defaultPassword").innerHTML+=" - <B>"+(typeof default_password!="undefined
"?default_password:"&ltWPS PIN&gt")+"</B>"}else{document.getElementById("defaultPassword").innerHTML+=" - <B>"+(typeof default_password!="undefined"?default_password:"admin")+"</B>"}if(INCLUDE_US_CPI_SPEC){document.
getElementById("defaultIpAddress").innerHTML+=" - <B>"+(typeof default_ip_address!="undefined"?default_ip_address:"192.168.15.15")+"</B>"}else{document.getElementById("defaultIpAddress").innerHTML+=" - <B>"+(typeof
default_ip_address!="undefined"?default_ip_address:"192.168.0.1")+"</B>"}document.getElementById("defaultSubnetMask").innerHTML+=" - <B>"+(typeof default_subnet_mask!="undefined"?default_subnet_mask:"255.255.255.0")
+"</B>";if(INCLUDE_FORBIDDEN_OLD_PWD){document.getElementsByClassName("admin")[0].style.display="none";document.getElementsByClassName("admin_0")[0].style.display="none";}</script>
_wr-841n.bin.extracted/squashfs-root/web/help/DynDdnsHelpRpm.htm:14:<p class="brief">The Router offers a Dynamic Domain Name System (<B>DDNS</B>) feature. DDNS lets you assign a fixed host and domain name to a dynam
lc Internet IP address. It is useful when you are hosting your own website, FTP server, or other server behind the Router. Before using this feature, you need to sign up with DDNS service providers such as <a href="
#" onClick="openWindow1();" class=L1>dyn.com</a>. The Dynamic DNS client service provider will give you a password or key.</P>
_wr-841n.bin.extracted/squashfs-root/web/help/DynDdnsHelpRpm.htm:19:<li class="password">Enter the <B>Password</B> for your DDNS account. </lI>
_wr-841n.bin.extracted/squashfs-root/web/help/DynDdnsHelpRpm.htm:25:<p class="notice"><B>Notice:</B>  If you want to login again with another account after a successful login, please click the <B>Logout</B
> button, then input your new username and password and click the <B>Login</B> button.</P>
_wr-841n.bin.extracted/squashfs-root/web/help/WlanConnectHelpRpm.htm:55:<li class="wep_password"> <B>Password</B> - Enter the matching WEP key information for your network</li>
_wr-841n.bin.extracted/squashfs-root/web/help/ManageControlHelpRpm.htm:24:in your browser. You will be asked for the Router's password. After successfully entering the password, you will be able to access the Router
's web-based utility.</p>
_wr-841n.bin.extracted/squashfs-root/web/help/ManageControlHelpRpm.htm:28:<li class="s_1">Be sure to change the Router's default password to a secure password.</li>
_wr-841n.bin.extracted/squashfs-root/web/help/WanDynamicIpV6CfgHelpRpm.htm:20:<li class="pppoev"><B>PPPoEv6</B> - Connections which use PPPoEv6 that requires a user name and password.</li>
_wr-841n.bin.extracted/squashfs-root/web/help/WanDynamicIpCfgHelpRpm.htm:25:<li class="pppoe_0"><B>PPPoE</B> - Connections which use PPPoE that requires a user name and password.</li>
ottoramaka@cloudshell:~/firmware_project$
```

```
/bin/mkdir -m 0777 -p /var/https
/bin/mkdir -m 0777 -p /var/lock
/bin/mkdir -m 0777 -p /var/log
/bin/mkdir -m 0777 -p /var/run
/bin/mkdir -m 0777 -p /var/tmp
/bin/mkdir -m 0777 -p /var/Wireless/RT2860AP
/bin/mkdir -m 0777 -p /var/tmp/wsc_upnp
cp -p /etc/SingleSKU_FCC.dat /var/Wireless/RT2860AP/SingleSKU.dat

/bin/mkdir -m 0777 -p /var/tmp/dropbear
```

```
sam@bonehead:~/midterm1/_wr-841n.bin.extracted/squashfs-root$ cat etc/passwd.bak
admin:$1$$iC.dUsGpxNNJGeOm1dFio/:0:0:root:/:/bin/sh
dropbear:x:500:500:dropbear:/var/dropbear:/bin/sh
nobody:*:0:0:nobody:/:/bin/sh
sam@bonehead:~/midterm1/_wr-841n.bin.extracted/squashfs-root$
sam@bonehead:~/midterm1/_wr-841n.bin.extracted/squashfs-root$ ls -la etc/passwd.bak
-rw-rw-r-- 1 sam sam 132 Apr 13  2021 etc/passwd.bak
sam@bonehead:~/midterm1/_wr-841n.bin.extracted/squashfs-root$ s
```

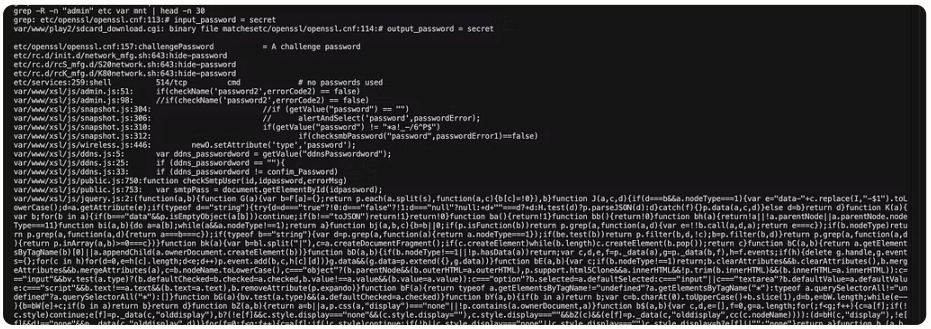
```
c/rc.d/rcK_mfg.d/K10lighttpd.sh:262:  echo "auth.require.accept.provision = $(admin-accept)" >> /tmp/lighttpd-inc.conf
c/rc.d/rcK_mfg.d/K10lighttpd.sh:404:  echo "auth.require.accept.provision = $(admin-accept)" >> /tmp/lighttpd-inc.conf
c/rc.d/rcK.d/K10lighttpd.sh:210:# admin always need auth
c/rc.d/rcK.d/K10lighttpd.sh:230:\$HTTP["url"] =~ "^/(.*/admin/|auth/|.*/mainFrame.cgi)" {
c/rc.d/rcK.d/K10lighttpd.sh:262:      echo "auth.require.accept.provision = $(admin-accept)" >> /tmp/lighttpd-inc.conf
c/rc.d/rcK.d/K10lighttpd.sh:404:      echo "auth.require.accept.provision = $(admin-accept)" >> /tmp/lighttpd-inc.conf
c/rc.d/rcS.d/S90lighttpd.sh:210:# admin always need auth
c/rc.d/rcS.d/S90lighttpd.sh:230:\$HTTP["url"] =~ "^/(.*/admin/|auth/|.*/mainFrame.cgi)" {
c/rc.d/rcS.d/S90lighttpd.sh:262:      echo "auth.require.accept.provision = $(admin-accept)" >> /tmp/lighttpd-inc.conf

c/rc.d/rcS.d/S90lighttpd.sh:404:      echo "auth.require.accept.provision = $(admin-accept)" >> /tmp/lighttpd-inc.conf
c/db/default.xml:140:<AdminUser type="3" content="admin" />
c/db/default.xml:141:<_DefAdmUser type="3" content="admin" />
c/dbus-1/system.conf:14:by a local sysadmin, create files matching
```

```
eSKU_FCC.dat
cat: /etc/SingleSKU_FCC.dat: No such file or directory
ottoramaka@cs-768527768013-default:~/firmware_project/_wr-841n.bin-0.extracted/squashfs-root$ grep -R -n "cp
 " etc/init.d | grep -v "#"
etc/init.d/rcS:15:cp -p /etc/SingleSKU_FCC.dat /var/Wireless/RT2860AP/SingleSKU.dat
etc/init.d/rcS:20:cp -p /etc/passwd.bak /var/passwd
ottoramaka@cs-768527768013-default:~/firmware_project/_wr-841n.bin-0.extracted/squashfs-root$ grep -R -n -E
"cp .*passwd|cp .*conf|cp .*dat|cp .*key|cp .*cfg" .
grep: ./usr/bin/dropbearmulti: binary file matches
grep: ./usr/bin/scp: binary file matches
grep: ./usr/bin/dropbear: binary file matches
grep: ./usr/bin/dropbearkey: binary file matches
```

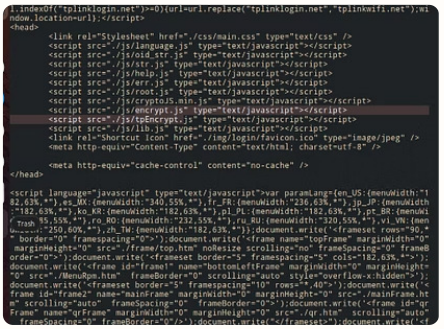
```
grep: ./usr/bin/arping: binary file matches
grep: ./usr/sbin/brctl: binary file matches
grep: ./bin/date: binary file matches
./etc/init.d/rcS:17:/bin/mkdir -m 0777 -p /var/tmp/dropbear
./etc/group:29:sshd:x:74:
./etc/services:45:ssh           22/tcp                          # SSH Remote Login Protocol
./etc/services:46:ssh           22/udp                          # SSH Remote Login Protocol
./etc/services:535:x11-ssh-offset       6010/tcp                        # SSH X11 forwarding offset
./etc/passwd.bak:2:dropbear:x:500:500:dropbear:/var/dropbear:/bin/sh
grep: ./bin/df: binary file matches
grep: ./bin/rm: binary file matches
grep: ./bin/mount: binary file matches
grep: ./bin/umount: binary file matches
grep: ./bin/echo: binary file matches
grep: ./bin/cat: binary file matches
```

Appendices: Screenshots and Evidence (Part 2)

Additional terminal outputs, blob images, and technical screenshots from the analysis.



```
grep -R -n "admin" etc var mnt | head -n 30
grep: etc/openssl/openssl.cnf:313:# input_password = secret
var/www/play2/sdcard_download.cgi: binary file matchesetc/openssl/openssl.cnf:114:# output_password = secret

etc/openssl/openssl.cnf:157:challengePassword            = A challenge password
etc/rc.d/init.d/network_mfg.sh:643:hide-password
etc/rc.d/rcS_mfg.d/S20network.sh:643:hide-password
etc/rc.d/rcK_mfg.d/K80network.sh:643:hide-password
etc/services:259:shell        514/tcp         cmd             # no passwords used
var/www/xsl/js/admin.js:51:     if(checkName('password2',errorCode2) == false)
var/www/xsl/js/admin.js:98:     //if(checkName('password2',errorCode2) == false)
var/www/xsl/js/snapshot.js:304:                         //if (getValue("password") == "")
var/www/xsl/js/snapshot.js:306:                         //      alertAndSelect('password',passwordError);
var/www/xsl/js/snapshot.js:310:                         if(getValue("password") != "*a1_-/6^P$")
var/www/xsl/js/snapshot.js:312:                                 if(checksmbPassword("password",passwordError1)==false)
var/www/xsl/js/Wireless.js:446:         newO.setAttribute('type','password');
var/www/xsl/js/ddns.js:5:       var ddns_passwordword = getValue("ddnsPasswordword");
var/www/xsl/js/ddns.js:25:      if (ddns_passwordword == "*){
var/www/xsl/js/ddns.js:33:      if (ddns_passwordword != confim_Password)
var/www/xsl/js/public.js:758:function checkSmtpUser(id,idpassword,errorMsg)
var/www/xsl/js/public.js:753:   var smtpPass = document.getElementById(idpassword);
var/www/xsl/js/jquery.js:2:function(a,b){function G(a){var b=F[a]={};return p.each(a.split(s),function(a,c){b[c]=!0})},b}function J(a,c,d){if(d===b&&a.nodeType===1){var e="data-"+c.replaceI,I,"-$1").toL
owerCase();d=a.getAttribute(e);if(typeof d=="string"){try{d=d=="true"?!0:d=="false"?!1:d=="null"?null:+d+""==d?+d:H.test(d)?p.parseJSON(d):d}catch(f){}p.data(a,c,d)}else d=b}return d}function K(a){
var b;for(b in a){if(b=="data"&&p.isEmptyObject(a[b]))continue;if(b!="toJSON")return!1}return!0}function bA(a){return!a||!a.parentNode||a.parentNode.node
Type===11}function bi(a,b){do a=a[b];while(a&&a.nodeType!==1);return a}function bj(a,b,c){b=b||0;if(p.isFunction(b))return p.grep(a,function(a,d){var e=!!b.call(a,d,a);return e===c});if(b.nodeType)retu
rn p.grep(a,function(a,d){return a===b===c});if(typeof b=="string"){var d=p.grep(a,function(a){return a.nodeType===1});if(be.test(b))return p.filter(b,d,!c);b=p.filter(b,d)}return p.grep(a,function(a,d
){return p.inArray(a,b)>=0===c})}function bk(a){var b=bl,split("|"),c=a.createDocumentFragment();if(c.createElement){while(b.length)c.createElement(b.pop());return c}function bC(a,b){return a.getElement
sByTagName(b)[0]||a.appendChild(a.ownerDocument.createElement(b))}function bD(a,b){if(b.nodeType===1||!p.hasData(a))return;var c,d,e,f=p._data(a),g=p._data(b,f),h=f.events;if(h){delete g.handle,g.event
s={};for(c in h)for(d=0,e=h[c].length;d<e;d++)p.event.add(b,c,h[c][d])}p.data&&(g.data=p.extend({},g.data))}function bE(a,b){var c;if(b.nodeType===1)return;b.clearAttributes&&b.clearAttributes(),b.merg
eAttributes&&b.mergeAttributes(a),c=b.nodeName.toLowerCase(),c=="object"?(b.parentNode&&(b.outerHTML=a.outerHTML),p.support.html5Clone&&a.innerHTML&&!p.trim(b.innerHTML)&&(b.innerHTML=a.innerHTML)):c=
=="input"&&bv.test(a.type)?(b.defaultChecked=b.checked=a.checked,b.value!==a.value&&(b.value=a.value)):c=="option"?b.selected=a.defaultSelected:c=="input"||c=="textarea"?b.defaultValue=a.defaultValu
e:c=="script"&&b.text!==a.text&&(b.text=a.text),b.removeAttribute(p.expando)}function bF(a){return typeof a.getElementsByTagName!="undefined"?a.getElementsByTagName("*"):typeof a.querySelectorAll!="un
defined"?a.querySelectorAll("*"):[]}function bG(a){bv.test(a.type)&&(a.defaultChecked=a.checked)}function bY(b,d){if(b in a)return b;var c=b.charAt(0).toUpperCase()+b.slice(1),ch,e=bW.length;while(e--
){b=bW[e]+c;if(b in a)return b}return d}function bZ(a,b){return a=b||a,p.css(a,"display")=="none"||!p.contains(a.ownerDocument,a)}function b$(a,b){var c,d,e=[],f=0,g=a.length;for(;f<g;f++){c=a[f];if(!
c.style)continue;e[f]=p._data(c,"olddisplay"),b?(!e[f]&&c.style.display=="none"&&(c.style.display=""),c.style.display===""&&bZ(c)&&(e[f]=p._data(c,"olddisplay",cc(c.nodeName)))):(dmH(c,"display"),e[
```



```
i:indexOf("tplinklogin.net")>=0){url=url.replace("tplinklogin.net","tplinkwifi.net");wi
ndow.location=url;</script>
<head>
        <link rel="Stylesheet" href="./css/main.css" type="text/css" />
        <script src="./js/language.js" type="text/javascript"></script>
        <script src="./js/oid_str.js" type="text/javascript"></script>
        <script src="./js/str.js" type="text/javascript"></script>
        <script src="./js/help.js" type="text/javascript"></script>
        <script src="./js/err.js" type="text/javascript"></script>
        <script src="./js/root.js" type="text/javascript"></script>
        <script src="./js/crypto35.min.js" type="text/javascript"></script>
        <script src="./js/encrypt.js" type="text/javascript"></script>
        <script src="./js/tpEncrypt.js" type="text/javascript"></script>
        <script src="./js/lib.js" type="text/javascript"></script>
        <link rel="Shortcut icon" href="./img/login/favicon.ico" type="image/jpeg" />
        <meta http-equiv="Content-Type" content="text/html; charset=utf-8" />

        <meta http-equiv="cache-control" content="no-cache" />

</head>

<script language="javascript" type="text/javascript">var paramLang={en_US:{menuWidth:"1
82,63%,*"},es_AV:{menuWidth:"340,55%,*"},fr_FR:{menuWidth:"236,63%,*"},ja_JP:{menuWidth
:"182,63%,*"},ko_KR:{menuWidth:"182,63%,*"},pl_PL:{menuWidth:"182,63%,*"},pt_BR:{menuWi
dth:"85,55%,*"},ro_RO:{menuWidth:"232,55%,*"},ru_RU:{menuWidth:"320,55%,*"},vi_VN:{men
uWidth:"259,60%,*"},zh_TW:{menuWidth:"182,63%,*"}};document.write('<frameset rows="90,*
" border="0" framespacing="0">');document.write('<frame name="topFrame" marginWidth="0"
marginHeight="0" src="./frame/top.htm" noResize scrolling="no" frameSpacing="0" frameB
order="0">');document.write('<frameset border="5" framespacing="5" cols="182,63%,*">');
document.write('<frame id="frame1" name="bottomLeftFrame" marginWidth="0" marginHeight=
"0" src="./MenuRpm.htm" frameBorder="0" scrolling="auto" style="overflow-x:hidden">');
document.write('<frameset border="5" framespacing="10" rows="*,40%">');document.write('<
frame id="frame2" name="mainFrame" marginWidth="0" marginHeight="0" src="./mainFrame.ht
m" scrolling="auto"  frameSpacing="0"  frameBorder="0">');document.write('<frame id="qr
Frame" name="qrFrame" marginWidth="0" marginHeight="0" src="./qr.htm" scrolling="auto"
frameSpacing="0" frameBorder="0"/>');document.write('</frameset>');document.write('<f
```

```
g_http_alias_conf_admin
g_http_author_admin
adminName
adminPwd
admin
<script type="text/javascript"> var authTimes=%d; var forbidTime=%d; var modelNa
me="%s"; var modelDesc="%s"; var forbidAdminLogin=%d; var forbidUserLogin=%d; va
r usernameIsAdmin=%d;</script>
adminName=%s
adminPwd=%s
```

```
t 0px -116px;
web/domain-redirect.htm:        background: url(img/login/domain-redirect.png) no-repea
t 0px 0px\0;
web/domain-redirect.htm:        background: url(img/login/domain-redirect.png) no-repea
t 0px -13px\0;
web/domain-redirect.htm:        background: url(img/login/domain-redirect.png) no-repea
t -14px 0px\0;
web/domain-redirect.htm:        background: url(img/login/domain-redirect.png) no-repea
t -14px -13px\0;
web/domain-redirect.htm:        background: url(img/login/domain-redirect.png) no-repea
```

```
cos &
sam@bonehead:~/midterm1/_wr-841n.bin.extracted/squashfs-root$ cat etc/passwd.bak
admin:$1$$iC.dUsGpxNNJGeOm1dFio/:0:0:root:/:/bin/sh
dropbear:x:500:500:dropbear:/var/dropbear:/bin/sh
nobody:*:0:0:nobody:/:/bin/sh
sam@bonehead:~/midterm1/_wr-841n.bin.extracted/squashfs-root$
sam@bonehead:~/midterm1/_wr-841n.bin.extracted/squashfs-root$ ls -la etc/passwd.bak
-rw-rw-r-- 1 sam sam 132 Apr 13  2021 etc/passwd.bak
sam@bonehead:~/midterm1/_wr-841n.bin.extracted/squashfs-root$ S
```

```
Created directory: /home/sam/snap/john-the-ripper/694/.john/opencl
Warning: detected hash type "md5crypt", but the string is also recognized as "md5crypt-long"
Use the "--format=md5crypt-long" option to force loading these as that type instead
Warning: detected hash type "md5crypt", but the string is also recognized as "md5crypt-opencl"
Use the "--format=md5crypt-opencl" option to force loading these as that type instead
Using default input encoding: UTF-8
Loaded 1 password hash (md5crypt, crypt(3) $1$ (and variants) [MD5 256/256 AVX2 8x3])
Will run 4 OpenMP threads
Note: Passwords longer than 5 [worst case UTF-8] to 15 [ASCII] rejected
fopen: usr/share/wordlists/rockyou.txt: No such file or directory
sam@bonehead:~/midterm1/_wr-841n.bin.extracted/squashfs-root$ python3 -c "import crypt; print(crypt.crypt('',
 '\$1\$\$'))"
```

References

- OSINT Framework. (n.d.). *Collection of OSINT Tools for Investigations and Research*. **https://osintframework.com**
- CWE-259: Use of Hard-coded Password. (2024, February 28). MITRE Common Weakness Enumeration. **https://cwe.mitre.org/data/definitions/259.html**
- CWE-732: Incorrect Permission Assignment for Critical Resource. (2024, February 28). MITRE Common Weakness Enumeration. **https://cwe.mitre.org/data/definitions/732.html**
- BusyBox. (n.d.). BusyBox: The Swiss Army Knife of Embedded Linux. **https://busybox.net**
- Binwalk. (n.d.). Firmware Analysis Tool. ReFirm Labs. **https://github.com/ReFirmLabs/binwalk**
- Nosey Parker. (n.d.). Sensitive Information Scanner for Source Code and Files. **https://github.com/praetorian-inc/noseyparker**
- OpenSSL Project. (n.d.). OpenSSL: Cryptography and SSL/TLS Toolkit. **https://www.openssl.org**
- TP-Link. (n.d.). TL-WR841N – Wireless N Router. **https://www.tp-link.com**
- D-Link. (n.d.). DCS-8000LH Mini HD Wi-Fi Camera. **https://www.dlink.com**